SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

Code of Conduct

I ___Y Sravan Kumar (192472289)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____Sravan Kumar_____

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3.An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4.A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5.A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.

Explain how message brokers, stream processing, and cloud storage are integrated.

- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

1. Media Company – Cloud Video Streaming Platform

Proposed Cloud Architecture

- Store videos in cloud object storage.
- Use a Content Delivery Network (CDN) to cache videos near users.
- Deploy application servers across multiple availability zones.
- Use load balancers to distribute incoming requests.
- Use auto-scaling to automatically increase or decrease computing resources.
- Use adaptive bitrate streaming so video quality can adjust according to the user's network speed.

Scalability and Caching

- Horizontal scaling adds more application/server instances when demand increases.
- CDN caching stores popular video content at edge locations closer to users.
- This reduces latency, buffering, and load on the origin servers.

Load Balancing and Auto-Scaling

- A load balancer distributes millions of user requests across available servers.
- Auto-scaling adds servers during peak traffic and removes unnecessary servers when traffic decreases.
- Health checks can stop traffic from being sent to failed servers.

Cost Optimization

- Use pay-as-you-go resources.
- Auto-scale down during low-traffic periods.
- Use lower-cost storage tiers for old or rarely accessed videos.
- Use CDN caching to reduce repeated requests to origin servers.
- Use reserved/committed capacity for predictable workloads and suitable discounted capacity for fault-tolerant background processing.

2. Fintech Startup – Containers Across Multiple Clouds

Proposed Solution

Use container orchestration such as Kubernetes to manage containerized applications across the environments.

Kubernetes can provide:

- Automated deployment
- Container scheduling
- Scaling
- Load balancing
- Self-healing
- Rolling updates

Using similar Kubernetes configurations across cloud providers improves deployment consistency.

Service Discovery

- Kubernetes provides built-in service discovery.
- Applications can communicate using stable service names rather than tracking individual container IP addresses.
- Internal load balancing can distribute requests among container replicas.

Scaling

- Horizontal Pod Autoscaling can automatically increase or decrease the number of application pods according to demand.
- Cluster/node scaling can add or remove computing capacity when required.

Benefits of Multi-Cloud

- Avoids excessive dependence on one cloud provider.
- Improves resilience.
- Provides geographic flexibility.
- Allows organizations to select suitable services from different providers.

Risks

- Increased management complexity.
- Different security and networking configurations.
- Higher data-transfer costs.
- Difficult monitoring and troubleshooting.
- Maintaining consistent compliance policies can be challenging.

3. E-Learning Platform – Backup and Disaster Recovery

Proposed Solution

Use three main techniques:

1. Cloud-native backups

- Schedule automatic backups of databases, files, and application data.
- Maintain backups separately from the production environment.

2. Replication

- Replicate important data to another availability zone or region.

- If the primary environment fails, the replica can be used for recovery.

3. Versioning

- Enable object/file versioning.
- When a file is accidentally deleted or modified, an older version can be restored.

RTO and RPO

RTO – Recovery Time Objective:

The maximum acceptable time required to restore the system after a failure.

Example:

RTO = 30 minutes means the service should be restored within 30 minutes.

RPO – Recovery Point Objective:

The maximum acceptable amount of data loss measured in time.

Example:

RPO = 5 minutes means the organization should lose no more than approximately five minutes of recent data.

Frequent replication reduces **RPO**, while automated failover and ready standby systems reduce **RTO**.

Business Continuity Steps

1. Perform automatic and regular backups.
2. Maintain copies in another region/account or isolated backup environment.
3. Enable versioning and appropriate retention policies.
4. Configure replication for critical data.
5. Create and document a disaster recovery plan.
6. Regularly test backup restoration and failover procedures.
7. Monitor backup and replication failures.

4. Government Agency – Hybrid Cloud

Proposed Architecture

```
Sensitive Applications/Data
        |
 On-Premises Datacenter
        |
 VPN / Dedicated Private Connection
        |
     Cloud
        |
Non-Sensitive Applications/Data
```

- Keep sensitive and compliance-regulated workloads on-premises.
- Place suitable non-sensitive and scalable applications in the public/private cloud.
- Connect both environments using a site-to-site VPN or dedicated private connection.
- Apply firewalls and network segmentation between systems.

Identity Management

- Use centralized Identity and Access Management (IAM).
- Implement federated identity/SSO where appropriate.
- Use Role-Based Access Control (RBAC).
- Apply the least-privilege principle.
- Require Multi-Factor Authentication (MFA) for sensitive operations.

Encryption

- Use encryption for data at rest.
- Use protocols such as TLS for data in transit.
- Securely manage encryption keys and rotate them periodically.

Challenges

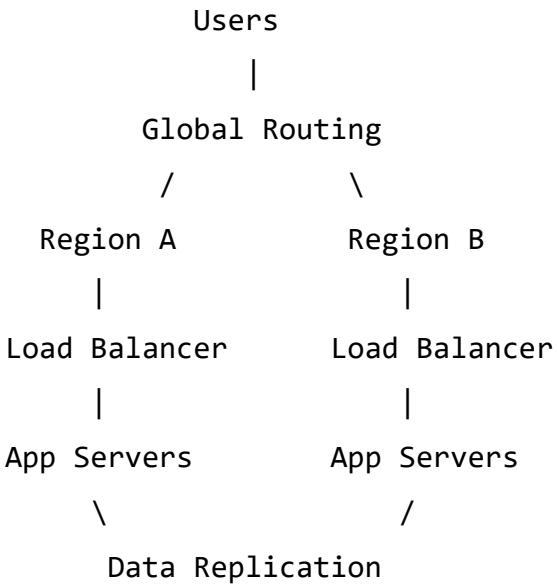
- Complex management across cloud and on-premises infrastructure.

- Maintaining consistent security policies.
- Compliance and auditing requirements.
- Network latency.
- Data synchronization difficulties.
- Increased monitoring and troubleshooting complexity.

5. SaaS Provider – High Availability and Fault Tolerance

Multi-Region Deployment

Deploy the application in at least two cloud regions.



If Region A fails, users can be redirected to Region B.

Data Replication

- Replicate critical application and database data between regions.
- Use synchronous replication where very low data loss is required and latency permits.
- Use asynchronous replication when lower latency and geographic distance are more important.
- Maintain backups in addition to replicas because replication alone does not protect against every type of data corruption or deletion.

Failover Mechanism

- Continuously perform health checks.
- Detect when the primary region becomes unavailable.
- Automatically route users to the healthy region.
- Use active-active deployment when both regions serve users simultaneously, or active-passive when the secondary region acts as a standby.

Monitoring and Alerting

- Monitor CPU, memory, network, database performance, latency, error rate, and application availability.
- Configure automated health checks.
- Create alerts when metrics cross predefined thresholds.
- Centralize application and infrastructure logs.
- Use dashboards to observe system health.
- Notify administrators immediately when critical failures occur.

Conclusion

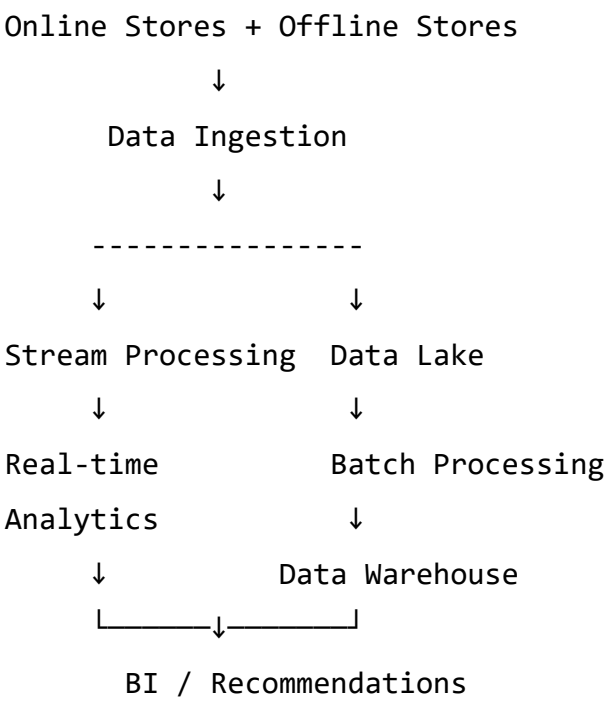
A multi-region architecture with replication, automated failover, load balancing, backups, health checks, and continuous monitoring provides high availability and allows the SaaS application to continue operating even when an entire region becomes unavailable.

SET 2 – Scenario-Based Questions

1. Retail Company – Cloud-Based Big Data Architecture

The company generates terabytes of transaction data from online and offline stores. A scalable big-data architecture can be used to process this information and generate near-real-time insights.

Proposed Architecture



Distributed Storage and Processing

- Use cloud object storage/data lake to store large volumes of raw transaction data.
- Use distributed frameworks such as Apache Spark for large-scale processing.
- Partition data across multiple nodes to enable parallel processing.

Batch and Stream Processing

- Batch processing analyzes large volumes of historical data at scheduled intervals.
- Stream processing analyzes transactions as they arrive.
- Stream processing can provide real-time recommendations and stock alerts, while batch processing can identify long-term purchasing patterns.

Data Lake + Data Warehouse

- Data Lake: Stores large amounts of raw structured, semi-structured, and unstructured data.
- Data Warehouse: Stores cleaned and structured data optimized for reporting and analytics.
- Data can therefore flow as: Sources → Data Lake → Processing → Data Warehouse → Analytics.

Scalability and Cost Efficiency

- Use auto-scaling for processing clusters.
- Use distributed/parallel processing.
- Partition data for faster access.
- Compress large datasets.
- Move old data to cheaper storage tiers.
- Use serverless or pay-as-you-go analytics where suitable.

2. Healthcare Organization – Secure Cloud Architecture

The healthcare organization needs strong security because patient records and medical images contain sensitive information.

Proposed Architecture

```
Doctors / Authorized Users
          ↓
    MFA + IAM
          ↓
 Secure Application/API
          ↓
   Encrypted Cloud
     Storage/Database
          ↓
 Backup + Audit Logs
```

Encryption

- Encrypt data at rest in databases, backups, and object storage.
- Encrypt data in transit using TLS.
- Sensitive patient information should never be transmitted through unsecured connections.

Identity and Access Management (IAM)

- Give each user a unique identity.
- Use Role-Based Access Control (RBAC).
- Doctors should access only records required for their work.
- Follow the least-privilege principle.
- Enable Multi-Factor Authentication (MFA).

Key Management

- Store encryption keys in a dedicated Key Management Service (KMS) rather than directly in application code.
- Rotate encryption keys regularly.
- Restrict access to keys.
- Maintain secure backup/recovery procedures for keys.

Auditing and Logging

Record important activities such as:

- Login attempts
- Patient-record access
- Data modifications
- File downloads
- Administrative actions

Centralized logs can be continuously analyzed for suspicious behavior and compliance violations.

Preventing Unauthorized Access

- MFA
- Strong IAM policies
- Encryption
- Firewalls and private networking
- Regular vulnerability scanning and patching
- Intrusion/threat detection

- Data-loss prevention controls
- Regular security audits

3. Online Ticket Booking – Serverless Architecture

A serverless architecture is suitable because ticket traffic can increase dramatically when popular events are announced.

Proposed Architecture

```
Users
  ↓
API Gateway
  ↓
Serverless Functions
  ↓
-------------------------
↓            ↓          ↓
Database   Queue     Object Storage
```

Event-Driven Computing

Serverless functions execute when events occur, such as:

- User searches for tickets
- User starts a booking
- Payment is completed
- Confirmation needs to be sent

During traffic spikes, additional function instances can automatically execute without manually provisioning servers.

API Gateway

- Receives client/API requests.

- Routes requests to serverless functions.
- Can provide authentication, throttling, and request control.

Managed Database

Stores:

- User information
- Events
- Seat availability
- Reservations
- Transactions

A managed database reduces administrative work such as patching and backups.

Message Queues

Message queues can handle operations asynchronously.

For example:

```
Booking → Queue → Payment/Confirmation Processing
```

This prevents sudden request spikes from overwhelming downstream systems.

Advantages

- Automatic scaling
- Reduced server management
- Pay for actual execution
- Good for unpredictable workloads
- Fast application development

Limitations

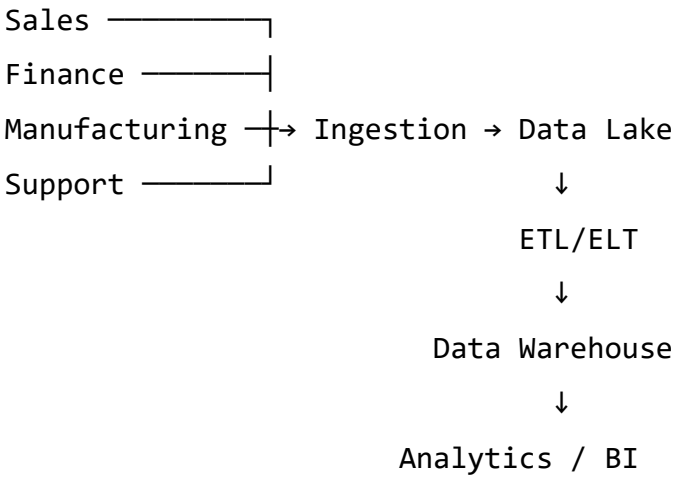
- Cold-start latency can occur.
- Execution time/resource limits may apply.

- Debugging distributed functions can be difficult.
- Vendor lock-in may increase.
- Costs can become high for continuously running, extremely heavy workloads.

4. Multinational Company – Cloud ETL/ELT Pipeline

The organization needs to integrate information from sales, finance, manufacturing, and customer-support systems.

Proposed Architecture



ETL and ELT

ETL:

Extract → Transform → Load

Data is extracted from source systems, cleaned/transformed, and then loaded into the target system.

ELT:

Extract → Load → Transform

Raw data is first loaded into scalable cloud storage/warehouse and transformations are performed afterward.

Data Quality

Maintain quality by:

- Removing duplicates
- Handling missing values
- Validating data types
- Standardizing formats
- Checking accuracy and consistency

Metadata Management

Metadata describes information about the data, including:

- Data source
- Schema
- Owner
- Creation/update time
- Transformation history

A centralized data catalog helps users discover and understand datasets.

Data Governance

- Define access-control policies.
- Assign data owners/stewards.
- Classify sensitive information.
- Maintain retention policies.
- Monitor regulatory compliance.

Workflow Orchestration

Workflow orchestration tools automate the order of pipeline operations.

For example:

```
Extract
   ↓
Validate
   ↓
Transform
   ↓
Load
   ↓
Quality Check
```

Scheduling tools can execute pipelines hourly, daily, or when new data arrives.

Performance and Cost Optimization

- Partition large datasets.
- Use columnar storage formats.
- Compress data.
- Cache frequently accessed data.
- Archive old data to lower-cost storage.
- Auto-scale processing resources.
- Shut down temporary processing resources when not required.

5. Logistics Company – Cloud ML and Analytics Architecture

The company wants to predict delivery delays using historical shipment data, weather information, and live GPS data.

Proposed Architecture

```
Historical Data + Weather + Live GPS
                ↓
          Data Ingestion
                ↓
             Data Lake
```

```
          ↓
  Big Data Processing
          ↓
   Feature Engineering
          ↓
     ML Model Training
          ↓
     Model Deployment
          ↓
Real-Time Prediction API
          ↓
Monitoring + Retraining
```

Big Data for Training and Inference

Distributed technologies such as Apache Spark can process large shipment datasets across multiple machines.

Historical information can be used for model training, while streaming GPS and weather information can be processed for real-time inference.

Cloud-Native AI/ML Services

Managed ML platforms can provide:

- Model training
- Experiment tracking
- Model registry/versioning
- Deployment endpoints
- Auto-scaling
- Performance monitoring

The trained model can be deployed as an API that receives current shipment information and predicts whether a delivery will be delayed.

Large-Scale Data Ingestion

- Use batch ingestion for historical shipment records.
- Use streaming ingestion for GPS information.
- Connect to weather-data APIs/services.
- Store raw information in a scalable data lake.

Feature Engineering

Useful features might include:

- Distance remaining
- Vehicle speed
- Traffic conditions
- Weather conditions
- Previous delivery delays
- Route characteristics
- Time of day

These features are transformed into suitable inputs for the ML model.

Model Updates

- Continuously monitor model accuracy.
- Detect data/model drift.
- Collect new delivery outcomes.
- Retrain the model periodically or when performance drops.
- Maintain model versions.
- Test a new model before replacing the production model.

Conclusion

A combination of cloud storage, streaming ingestion, distributed big-data processing, feature engineering, managed ML services, real-time inference, monitoring, and automated retraining enables the logistics company to predict delivery delays at large scale.

SET 3 – Scenario-Based Questions

1. Manufacturing Company – Cloud Migration Strategy

The manufacturing company needs to migrate its legacy applications and databases from an on-premises data center to the cloud with minimum downtime, good performance, and strong security.

Proposed Migration Strategy

The migration can be performed in phases:

Assessment → Planning → Pilot Migration → Data Migration → Application Migration → Testing → Final Cutover → Monitoring

Before migration, identify application dependencies, database size, security requirements, performance requirements, and acceptable downtime.

Six Migration Approaches

1. Rehost – "Lift and Shift"

- Move the existing application to cloud virtual machines with minimal changes.
- Suitable for applications that need to be migrated quickly.
- Requires less development effort.

2. Replatform – "Lift, Tinker and Shift"

- Move the application while making small optimizations.
- For example, migrate an existing database to a managed cloud database.
- Reduces administration without completely redesigning the application.

3. Refactor

- Redesign the application to take advantage of cloud-native technologies.
- Applications may be divided into microservices or containers.
- Provides better scalability and performance but requires more development effort.

4. Repurchase

- Replace an existing legacy application with a cloud/SaaS product.
- Useful when maintaining the old application is expensive.

5. Retire

- Remove applications that are no longer required.
- Reduces migration effort and cloud costs.

6. Retain

- Keep some applications on-premises.
- Suitable when applications cannot be migrated because of compliance, technical, or business requirements.

Performance Validation

After migration:

- Perform load and stress testing.
- Compare cloud performance against the original on-premises baseline.
- Measure CPU, memory and storage utilization.
- Measure application response time and latency.
- Test database query performance.
- Test applications under peak workloads.
- Continuously monitor errors and availability.

Reducing Migration Risk and Downtime

- Perform migration in phases instead of migrating everything simultaneously.
- Create complete backups before migration.
- Replicate databases continuously before final cutover.

- Perform pilot migrations first.
- Test applications in a staging environment.
- Schedule final cutover during low-traffic periods.
- Maintain a rollback plan.
- Use encryption and strong IAM policies.
- Monitor applications continuously after migration.

2. Smart City – Cloud IoT Architecture

Thousands of IoT sensors generate continuous data about traffic, air quality, energy consumption and public safety.

Proposed Architecture

```
IoT Sensors
     ↓
IoT Gateway
     ↓
Message Broker
     ↓
Stream Processing
   ↙       ↘
Alerts    Cloud Storage
             ↓
        Data Analytics
             ↓
         Dashboard
```

Message Brokers

A message broker receives sensor messages and distributes them to processing systems.

It helps:

- Handle large numbers of sensor messages.
- Buffer sudden traffic spikes.
- Decouple sensors from processing applications.
- Improve reliability.

Technologies such as MQTT and Apache Kafka can support IoT messaging architectures.

Stream Processing

Stream-processing systems process sensor information immediately as it arrives.

For example:

Traffic sensor → Detect congestion → Generate alert → Update traffic control system

This enables real-time decision-making.

Cloud Storage

Different storage can be used for different requirements:

- Real-time databases for current sensor states.
- Object storage/data lakes for historical sensor information.
- Data warehouses for analytics and reporting.

Handling High-Volume and High-Velocity Data

- Partition sensor streams.
- Process data in parallel.
- Use distributed stream-processing systems.
- Perform edge processing to filter unnecessary data before sending it to the cloud.
- Batch/compress sensor data where appropriate.

Scalability, Reliability and Security

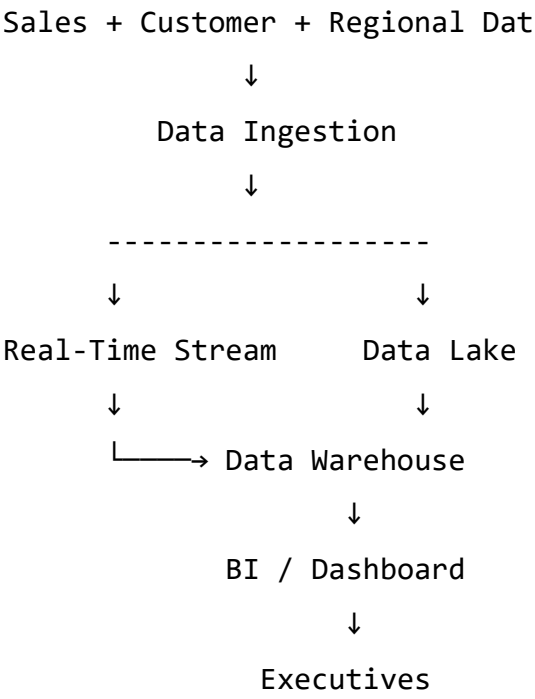
- Use auto-scaling.
- Deploy services across multiple availability zones.

- Use message queues/brokers to prevent data loss.
- Encrypt data in transit and at rest.
- Authenticate IoT devices.
- Use IAM and least-privilege access.
- Monitor devices and services continuously.

3. E-Commerce Company – Cloud Business Intelligence

The company requires dashboards showing sales, customer behavior, and regional trends using both real-time and historical information.

Proposed Architecture



Data Visualization and Data Warehouse

The cloud data warehouse stores cleaned and structured business information.

BI visualization tools connect to the warehouse and create:

- Sales charts

- Revenue reports
- Customer-behavior dashboards
- Regional comparisons
- KPI dashboards

Real-time pipelines can continuously update recent information, while historical warehouse data supports long-term reporting.

Dashboard Performance Optimization

- Partition large tables.
- Use indexes where supported and appropriate.
- Use columnar storage.
- Create materialized views or pre-aggregated tables.
- Cache frequently requested results.
- Avoid loading unnecessary columns.
- Optimize SQL queries.
- Use incremental refresh instead of refreshing the entire dataset.

Security Measures

- Implement Role-Based Access Control (RBAC).
- Use Multi-Factor Authentication (MFA).
- Encrypt data at rest and in transit.
- Apply row-level or column-level security for sensitive information.
- Maintain audit logs.
- Mask sensitive customer information.
- Follow the least-privilege principle.

4. Online Gaming Company – Cloud Monitoring and Performance

The gaming company experiences varying workloads and high latency during peak hours.

Proposed Monitoring Architecture

```
Gaming Application
       ↓
Metrics + Logs + Traces
       ↓
Cloud Monitoring System
       ↓
Dashboards & Alerts
       ↓
Auto-Scaling / Optimization
```

Metrics

Monitor:

- CPU utilization
- Memory utilization
- Network latency
- Request rate
- Error rate
- Database response time
- Active users

Metrics help determine whether the infrastructure has sufficient resources.

Logs

Logs record application and system events such as:

- Errors
- Failed requests
- Authentication problems
- Server failures

Centralized logging makes troubleshooting easier.

Distributed Tracing

Tracing follows a user request across different application components or microservices.

For example:

User → Game server → API → Database

It helps identify which component is causing high latency.

Resource Optimization

- Use auto-scaling during peak hours.
- Scale down resources during low-demand periods.
- Use load balancing.
- Optimize database queries.
- Cache frequently requested data.
- Use CDN/edge locations for suitable static content.
- Perform capacity planning using historical usage patterns.

Availability and Cost Control

- Deploy across multiple availability zones.
- Use health checks and automatic failover.
- Use auto-scaling rather than permanently running peak capacity.
- Use committed/reserved capacity for predictable baseline workloads.
- Shut down unnecessary development/test resources.
- Continuously monitor cloud spending.

5. Financial Institution – Data Lifecycle Management

The financial institution stores petabytes of transaction information and must retain historical records while reducing storage costs.

Proposed Data Lifecycle

```
Transaction Data
      ↓
Classification
      ↓
Hot Storage
      ↓
Warm Storage
      ↓
Cold / Archive Storage
      ↓
Secure Deletion after
Retention Period
```

Data Classification

Classify information according to its importance and usage:

Hot Data

- Frequently accessed.
- Stored in high-performance storage.

Warm Data

- Accessed occasionally.
- Stored using lower-cost storage.

Cold/Archive Data

- Rarely accessed historical records.
- Stored in low-cost archival storage.

Retention Policies

Define how long each type of data must be retained.

For example:

Active data → Hot storage → Archive after defined period → Retain according to regulation → Secure deletion when legally permitted

Lifecycle policies can automatically move information between storage tiers.

Governance and Auditing

Data governance ensures that information is properly:

- Classified
- Stored
- Protected
- Accessed
- Retained
- Deleted

Auditing records who accessed or modified information and when it occurred.

For financial data, strong governance and audit trails help demonstrate compliance with applicable regulations.

Security

- Encrypt data at rest and in transit.
- Use IAM and Role-Based Access Control.
- Apply least-privilege access.
- Use MFA for sensitive administrative access.
- Protect encryption keys using a key-management system.
- Use immutable/WORM storage where regulations require records to be protected from modification.

Balancing Accessibility, Security and Cost

- Keep frequently used data in fast storage.
- Move old data to cheaper archival tiers.

- Use automated lifecycle policies.
- Compress large datasets.
- Partition data for efficient retrieval.
- Maintain searchable metadata/catalogs.
- Use encryption and strict access controls at every storage tier.

This provides fast access to important current information while keeping long-term historical records secure at a lower storage cost